

Responses ↕

Web search

Allow models to search the web for the latest information before generating a response.

[Copy Page](#)

Web search allows models to access up-to-date information from the internet and provide answers with sourced citations. To enable this, use the web search tool in the Responses API or, in some cases, Chat Completions.

There are three main types of web search available with OpenAI models:

1. **Non-reasoning web search:** The non-reasoning model sends the user's query to the web search tool, which returns the response based on top results. There's no internal planning and the model simply passes along the search tool's responses. This method is fast and ideal for quick lookups.
2. **Agentic search with reasoning models** is an approach where the model actively manages the search process. It can perform web searches as part of its chain of thought, analyze results, and decide whether to keep searching. This flexibility makes agentic search well suited to complex workflows, but it also means searches take longer than quick lookups. For example, you can adjust reasoning levels on models like `gpt-5.5` to change both the depth and latency of the search.
3. **Deep research** is a specialized, agent-driven method for in-depth, extended investigations by reasoning models. The model conducts web searches as part of its chain of thought, often tapping into hundreds of sources. Deep research can run for several minutes and is best used with background mode. Use `gpt-5.5` with reasoning set to `high` or `xhigh`.

Choose an integration

[Ask AI](#)

Loading docs agent...

```
3
4  const response = await client.responses.create({
5    model: "gpt-5.6",
6    tools: [{ type: "web_search" }],
7    input: "What was a positive news story from today?",
8  });
9
10 console.log(response.output_text);
```

Output and citations

Model responses that use the web search tool will include two parts:

- A `web_search_call` output item with the ID of the search call, along with the action taken in `web_search_call.action`. The action is one of:

Docs agent

Loading docs agent...

```
3     "type": "web_search_call",
4     "id":
5     "ws_67c9fa0502748190b7dd390736892e100be649c1a5ff9609",
6     "status": "completed",
7     "action": {
8         "type": "search",
9         "query": "latest news about AI"
10    }
11 },
12 {
13     "id":
14     "msg_67c9fa077e288190af08fdffda2e34f20be649c1a5ff9609",
15     "type": "message",
16     "status": "completed",
17     "role": "assistant",
```

Docs agent

Loading docs agent...

`external_web_access`, and
`return_token_budget`

`gpt-4o-search-preview`
or `gpt-4o-mini-search-`
`preview`

Migrate to Responses
`web_search`, or use
`gpt-5-search-api` if you
must stay on Chat
Completions

The preview search models are
deprecated and shut down on 2026-
07-23

Chat Completions search
integrations

Use `gpt-5-search-api`,
or migrate to Responses
`web_search` for more tool
controls and optional
search

Chat Completions search models
always search before responding;
Responses search is a tool

Docs agent

Loading docs agent...

default for most requests. Set it to `unlimited` only for high-effort research or evaluation runs that need to inspect many pages and might otherwise stop at the standard returned-token cap.

Use `unlimited` selectively because it can increase latency and cost. For long-running multi-search tasks, use background mode (`background: true`) so the request can keep running asynchronously and you can retrieve the final response later.

Value	Behavior
<code>default</code>	Uses the standard returned-token budget for web search results. This is the same behavior as omitting <code>return_token_budget</code> .

Docs agent

Loading docs agent...

Domain filtering

Domain filtering in web search lets you limit results to a specific set of domains. With the `filters` parameter you can configure up to 100 `allowed_domains` or up to 100 `blocked_domains`. When formatting domains, omit the HTTP or HTTPS prefix. For example, use `openai.com` instead of `https://openai.com/`. This approach also includes subdomains in the search. Note that domain filtering is only available in the Responses API with the `web_search` tool.

Sources

To view all URLs retrieved during a web search, use the `sources` field. Unlike inline citations, which show only the most relevant references, sources returns the complete list of URLs the model consulted when forming its response. The number of sources is often greater than the number of citations. Real-time third-party feeds are

Docs agent

Loading docs agent...

```
25     ],  
26     "tool_choice": "auto",  
27     "include": ["web_search_call.action.sources"],  
28     "input": "Please perform a web search on how semaglutide is used  
29     }'
```

Image search results

Web search can return image results alongside regular text results. Use image search when your application needs current or web-grounded visuals, such as product photos, landmarks, places, events, or visual references.

To use image search, set `search_content_types` to include `image`. Add `text` when you also want supporting text results that help the model summarize, rank, or explain the retrieved images.

Docs agent

Loading docs agent...

```
18   input:
19     "Search for recent images and supporting text sources about the
20   });
21
22   console.log(response.output);
```

Each `image_result` includes:

- `image_url`: The canonical image URL for the result.
- `source_website_url`: The page where the image was found.
- `thumbnail_url`: A thumbnail URL when available.
- `caption`: A short caption or description when available.



Docs agent

Loading docs agent...

To refine search results based on geography, you can specify an approximate user location using country, city, region, and/or timezone.

- The `city` and `region` fields are free text strings, like `Minneapolis` and `Minnesota` respectively.
- The `country` field is a two-letter ISO country code, like `US`.
- The `timezone` field is an IANA timezone like `America/Chicago`.

 Note that user location is not supported for deep research models using web search.

Customizing user location

JavaScript  

```
1 import OpenAI from "openai";
2 const openai = new OpenAI();
3
```

Docs agent

Loading docs agent...

```
1  curl "https://api.openai.com/v1/responses" \
2    -H "Content-Type: application/json" \
3    -H "Authorization: Bearer $OPENAI_API_KEY" \
4    -d '{
5      "model": "gpt-5.6",
6      "tools": [
7        { "type": "web_search", "external_web_access": false }
8      ],
9      "tool_choice": "auto",
10     "input": "Find when the Eiffel Tower opened to the public and ci
11   }'
```

Limitations

Chat Completions API

Docs agent

Loading docs agent...

Model context		
Model	window	Limitation
gpt-4.1	1M	Search context is limited to 128k
gpt-4.1-mini	1M	Search context is limited to 128k
o4-mini	200k	Search context is limited to 128k; <u>deprecated, shutdown 2026-10-23</u>

For Responses API web search, the search context window is limited to 128k, even when the model context window is larger.

- Web search does not support `gpt-5` with `minimal` reasoning.
- `gpt-5.4` with reasoning effort set to `none` may produce lower-quality results.

Docs agent

Loading docs agent...